

28.10.2024

Сортировки. Реальные примеры для сортировок.

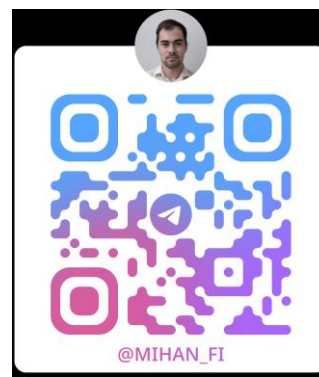
Филиппов Михаил Витальевич

m.filippov@g.nsu.ru

89232283872

Императивное программирование, 2025-2026

N * Новосибирский
государственный
университет
***НАСТОЯЩАЯ НАУКА**

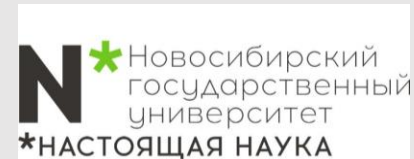


Давайте познакомимся

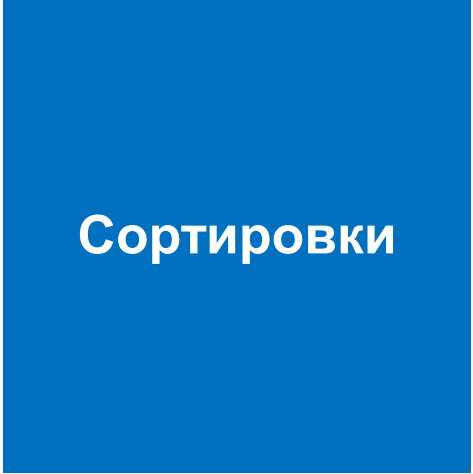


Филиппов Михаил Витальевич

- Окончил магистратуру ФФ НГУ
- Окончил аспирантуру ИТ СО РАН
- Backend C++ разработчик в Яндекс.
- м.н.с. ИТ СО РАН
- 8+ лет опыт в программировании C/C++



Структура лекции



Сортировки

90 минут

Терминология

Коллекция A элементов, которые можно сравнивать один с другим, сортируется на месте (без привлечения дополнительной памяти). Мы используем обозначения $A[i]$ и a_i для обозначения i -го элемента коллекции. По соглашению первым элементом коллекции является $A[0]$. Мы используем запись $A[\text{low}, \text{low}+n)$ для обозначения подколлекции $A[\text{low}] \dots A[\text{low}+n-1]$ из n элементов.

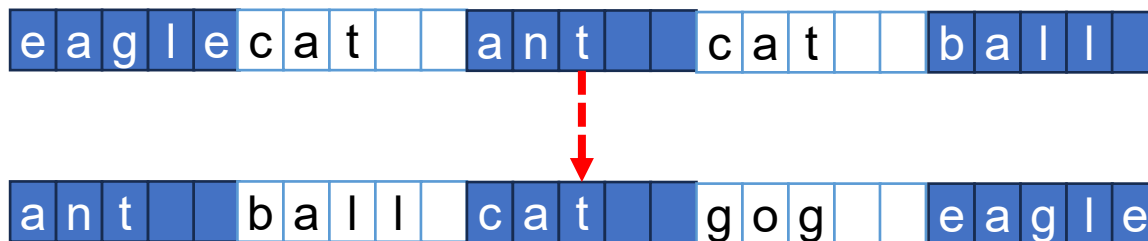
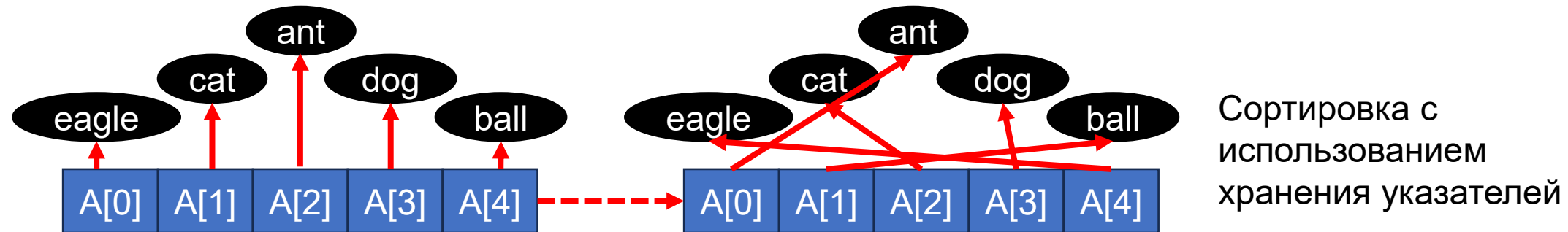
Для сортировки коллекции необходимо реорганизовать элементы A так, что если $A[i] < A[j]$, то $i < j$. Если в коллекции имеются повторяющиеся элементы, то они должны быть смежными в результирующей упорядоченной коллекции; т.е. если в отсортированной коллекции $A[i] = A[j]$, то не может быть такого k , что $i < k < j$ и $A[i] \neq A[k]$. Наконец отсортированная коллекция A должна представлять собой перестановку элементов, которые первоначально образовывали коллекцию A .



Представление

Коллекция может храниться в оперативной памяти компьютера, но может находиться и в файле в файловой системе, известной как вторичная память. Коллекция может быть заархивирована в третичной памяти (например, ленточные библиотеки или оптические диски), что может потребовать дополнительного времени только для обнаружения информации; кроме того, прежде чем она сможет быть обработана, возможно, потребуется скопировать такую информацию во вторичную память (например, на жесткий диск).

Информация, хранящаяся в оперативной памяти, обычно принимает одну из двух форм: на основе указателей или на основе значений.



Сравнимость элементов

Элементы в наборе данных сравнений должны обеспечивать нестрогое упорядочение. То есть для любых двух элементов p и q из набора данных должен выполняться ровно один из следующих трех предикатов: $p=q$, $p<q$ или $p>q$.

Распространенные сортируемые типы-примитивы включают целые числа, числа с плавающей точкой и символы. При сортировке составных элементов на каждую отдельную часть составного элемента накладывается лексикографическое упорядочивание, сводя, таким образом, сложную сортировку к отдельным сортировкам примитивных типов.

Вопрос упорядочения далеко не так прост при рассмотрении регистров символов (“А” больше или меньше “а?”), диакритических знаков (как соотносятся “а” и “ä”?) и дифтонгов (то же для “а” и “æ”?).

Будем предполагать, что вы можете предоставить компаратор — функцию сравнения `cmp`, которая сравнивает элементы p и q и возвращает 0, если $p=q$, отрицательное число, если $p<q$, и положительное число при $p>q$.



Устойчивая сортировка

Когда функция сравнения стр определяет, что два элемента исходной неупорядоченной коллекции a_i и a_j равны, может оказаться важным сохранить их относительный порядок в отсортированном множестве. Алгоритмы сортировки, которые гарантируют выполнение этого свойства, называются устойчивыми.

Город назначения	Компания	Рейс	Время вылета (в порядке возрастания)	Город назначения	Компания	Рейс	Время вылета (в порядке возрастания)
Buffalo	Air Trans	549	10:42	Atlanta	Southwest	482	13:20
Atlanta	Delta	1097	11:00	Atlanta	Delta	1097	11:00
Baltimore	Southwest	836	11:05	Atlanta	Air Trans	872	11:15
Atlanta	Air Trans	872	11:15	Atlanta	Delta	28	12:00
Atlanta	Delta	28	12:00	Atlanta	Al Italia	3429	13:50
Boston	Delta	1056	12:05	Austin	Southwest	1045	13:05
Baltimore	Southwest	216	12:20	Baltimore	Southwest	836	11:05
Austin	Southwest	1045	13:05	Baltimore	Southwest	216	12:20
Albany	Southwest	482	13:20	Baltimore	Southwest	272	13:40
Boston	Air Trans	515	13:21	Boston	Delta	1056	12:05
Baltimore	Southwest	272	13:40	Boston	Air Trans	515	13:21
Atlanta	Al Italia	3429	13:50	Boston	Air Trans	549	10:42

Критерии выбора алгоритма сортировки

Критерий	Алгоритм сортировки
Только несколько элементов	Сортировка вставками
Элементы уже почти отсортированы	Сортировка вставками
Важна производительность в наихудшем случае	Пирамидальная сортировка
Важна производительность в среднем случае	Быстрая сортировка
Элементы с равномерным распределением	Блочная сортировка
Как можно меньший код	Сортировка вставками
Требуется устойчивость сортировки	Сортировка слиянием



Сортировки перестановкой: пузырьковая

5	3	2	1	4
---	---	---	---	---

3	2	1	4	5
---	---	---	---	---

2	1	3	4
---	---	---	---

1	2	3
---	---	---

Наилучший случай = $N - 1$;
Наихудший случай =
арифметическая прогрессия =
$$N - 1 + \dots + 1 = \frac{(1+N-1)N}{2} = \frac{N^2}{2} =$$
$$O(N^2)$$



Сортировки перестановкой: пузырьковая

```
void sortBubble(void **ar, int n, int (*cmp)(const void *, const void *))
{
    _Bool swappedElements = 1;
    while (swappedElements)
    {
        n = n - 1;
        swappedElements = 0;
        for (int i = 1; i < n; i++)
        {
            if (cmp(ar[i], ar[i + 1]) > 0)
            {
                void *value = ar[i];
                ar[i] = ar[i + 1];
                ar[i + 1] = value;
                swappedElements = 1;
            }
        }
    }
}
```



Сортировки перестановкой: пузырьковая

Средний случай:

$C(i)$ число сравнений, выполненных на первых i проходах

Среднее число сравнений:

$$A(N) = \frac{1}{N-1} \sum_{i=1}^{N-1} C(i)$$

$$C(i) = \sum_{j=N-1}^i j = \frac{i+N-1}{2} (N-i) = \frac{N^2 - N - i^2 + i}{2}$$

$$\begin{aligned} A(N) &= \frac{1}{N-1} \sum_{i=1}^{N-1} \frac{N^2 - N - i^2 + i}{2} = \frac{N^2 - N}{2} - \frac{1}{2} \frac{1}{N-1} \sum_{i=1}^{N-1} i^2 + \frac{1}{2} \frac{1}{N-1} \frac{1+N-1}{2} N-1 = \\ &= \frac{N^2 - N}{2} + \frac{N}{4} - \frac{1}{2} \frac{1}{N-1} \frac{(N-1)N(2N-1)}{6} = \frac{(6-2)N^2 + (-6+3+1)N}{12} = \\ &= \frac{1}{3} N^2 \left(1 - \frac{1}{2N} \right) = O(N^2) \end{aligned}$$



Сортировки перестановкой: вставками

Основная идея сортировки вставками состоит в том, что при добавлении нового элемента в уже отсортированный список его стоит сразу вставлять в нужное место вместо того, чтобы вставлять его в произвольное место, а затем заново сортировать весь список.

5	3	2	1	4
---	---	---	---	---

3	5	2	1	4
---	---	---	---	---

2	3	5	1	4
---	---	---	---	---

1	2	3	5	4
---	---	---	---	---

1	2	3	4	5
---	---	---	---	---

Наилучший случай = $N - 1$;
Наихудший случай =
арифметическая прогрессия =
$$N - 1 + \dots + 1 = \frac{(1+N-1)N}{2} = \frac{N^2}{2} = O(N^2)$$



Сортировки перестановкой: вставками

```
void sortPointers(void **ar, int n, int (*cmp)(const void *, const void *))
{
    int j;
    for (j = 1; j < n; j++)
    {
        int i = j - 1;
        void *value = ar[j];
        while (i >= 0 && cmp(ar[i], value) > 0)
        {
            ar[i + 1] = ar[i];
            i--;
        }
        ar[i + 1] = value;
    }
}
```



Сортировки перестановкой: вставками

Средний случай:

Сколько существует возможных положений для i -ого элемента? $i + 1$

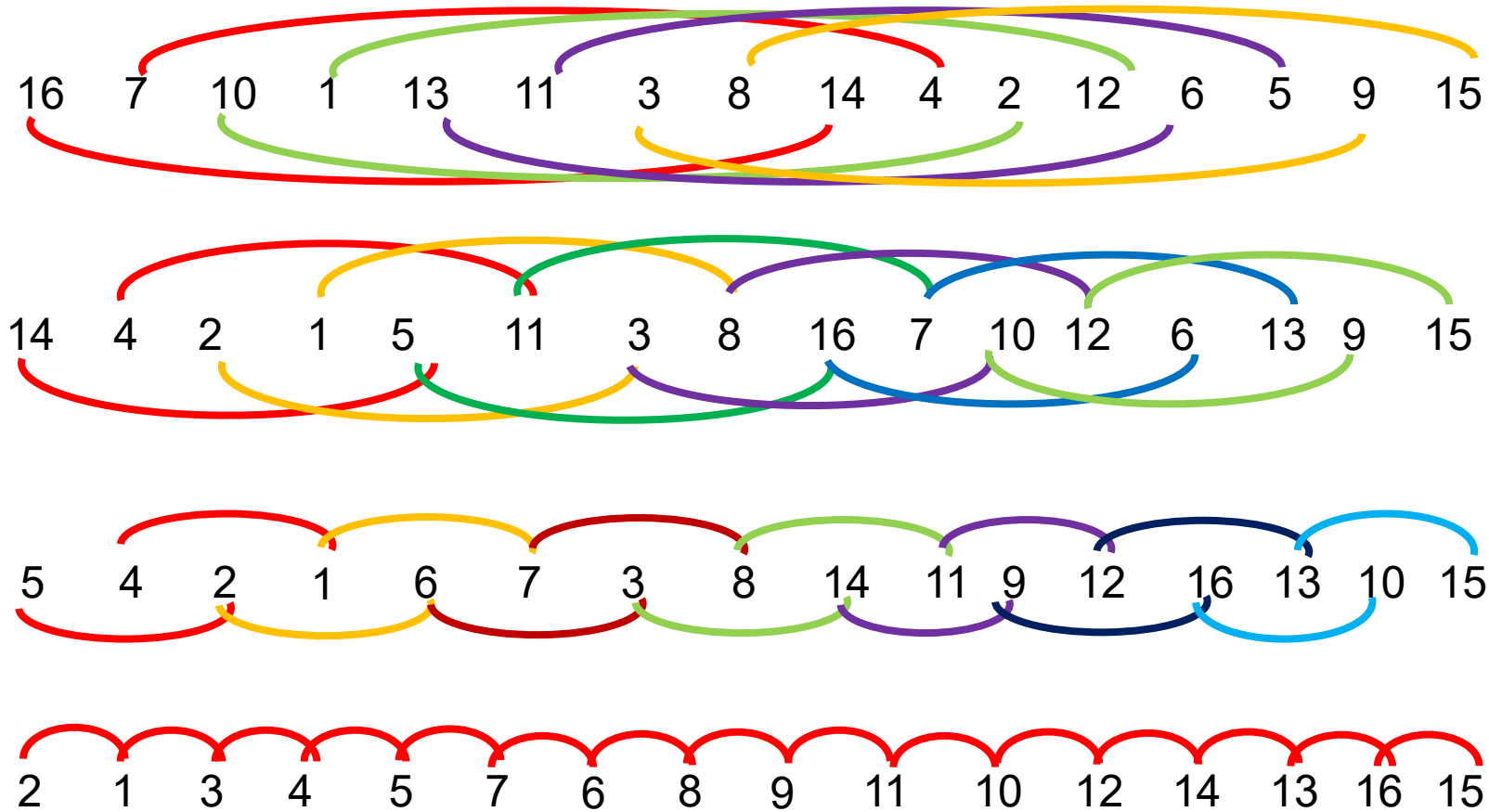
$A(i)$ среднее число сравнений для вставки i -го элемента

Среднее число сравнений:

$$\begin{aligned} A(i) &= \frac{1}{i+1} \sum_{p=1}^i p + i = \frac{1}{i+1} \left(\frac{i(i+1)}{2} + i \right) = \frac{i}{2} + 1 - \frac{1}{i+1} \\ A(N) &= \sum_{i=1}^{N-1} A(i) = \sum_{i=1}^{N-1} \left(\frac{i}{2} + 1 - \frac{1}{i+1} \right) = \frac{1}{2} \frac{N(N-1)}{2} + N - 1 - \sum_{i=1}^N \frac{1}{i} + 1 = \\ &= \frac{N^2 + 3N}{4} - \ln(N) = \frac{N^2}{4} \left(1 + \frac{3}{4N} - \frac{\ln(N)}{4N^2} \right) = O(N^2) \end{aligned}$$



Сортировка Шелла



Сортировка Шелла

```
void sortShell(void **ar, int n, int (*cmp)(const void *, const void *))
{
    int j;
    int passes = log(n) / log(2);
    while (passes >= 1)
    {
        int increment = pow(2, passes) - 1;
        for (int i = 1; i < increment; i++)
        {
            InsertionSort(*ar, n, i, increment);
        }
        passes--;
    }
}
```

Было доказано, что сложность этого алгоритма в наихудшем случае при выбранных нами значениях шага равна $O(N^{3/2})$



Корневая сортировка

Исходный список

310 213 023 130 013 301 222 032 201 111 323 002 330 102 231 120

Номер стопки	Содержимое			
0	310	130	330	120
1	301	201	111	231
2	222	032	002	102
3	213	023	013	323

Номер стопки	Содержимое			
0	301	201	002	102
1	310	111	213	013
2	120	222	023	323
3	130	330	231	032

Номер стопки	Содержимое			
0	002	013	023	032
1	102	111	120	130
2	201	213	222	231
3	301	310	323	330

(а) Первый проход, разряд единиц

Список, полученный в результате первого прохода

310 130 330 120 301 201 111 231 222
032 002 102 213 023 013 323

(б) Второй проход, разряд десятков

Список, полученный в результате второго прохода

301 201 002 102 310 111 213 013 120
222 023 323 130 330 231 032

(в) Третий проход, разряд сотен



Корневая сортировка

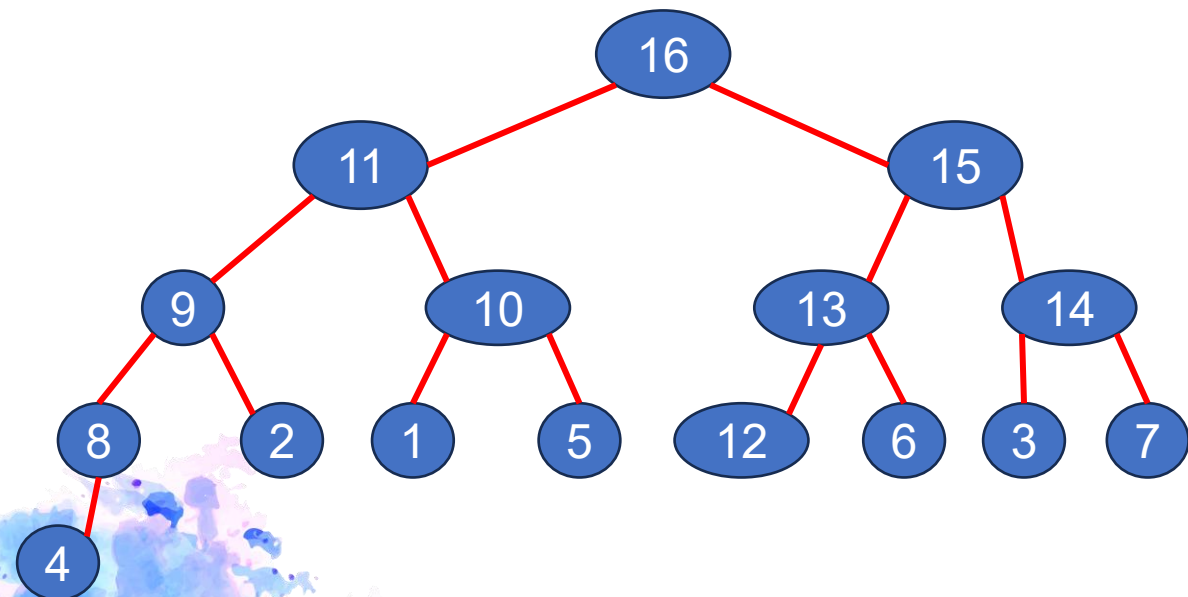
```
void sortRadix(void **ar, int n, int (*cmp)(const void *, const void *))
{
    int shift = 1;
    for (int loop = 1; loop < keySize; loop++)
    { // keySize - количество ключей
        for (int entry = 1; entry < N; entry++)
        {
            int bucketNumber = (ar[entry].key / shift) % 10;
            Append(bucket[bucketNumber], ar[entry]);
        }
        ar = CombineBuckets();
        shift *= 10;
    }
}
```

- + Скорость $O(NM)$, M – количество различных символов
- Много требует памяти



Пирамидальная сортировка

В основе пирамидальной сортировки лежит специальный тип бинарного дерева, называемый пирамидой



01	02	03	04	05	06	07	16	09	10	11	12	13	14	15	08
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

01	02	03	04	05	06	15	16	09	10	11	12	13	14	07	08
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

01	02	03	04	05	13	15	16	09	10	11	12	06	14	07	08
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

01	02	03	04	11	13	15	16	09	10	05	12	06	14	07	08
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

01	02	03	16	11	13	15	08	09	10	05	12	06	14	07	04
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

01	02	15	16	11	13	14	08	09	10	05	12	06	03	07	04
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

01	16	15	09	11	13	14	08	02	10	05	12	06	03	07	04
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

16	11	15	09	10	13	14	08	02	01	05	12	06	03	07	04
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

Пирамидальная сортировка (1/2)

```
static void heapify(void **ar, int (*cmp)(const void *, const void *), int idx, int max)
{
    int left = 2 * idx + 1;
    int right = 2 * idx + 2;
    int largest; /* Поиск максимального элемента среди A[idx], A[left] и A[right]. */
    if (left < max && cmp(ar[left], ar[idx]) > 0) {
        largest = left;
    }
    else {
        largest = idx;
    }
    if (right < max && cmp(ar[right], ar[largest]) > 0) {
        largest = right;
    }
    /* Если наибольший элемент не является родительским, выполняем обмен и переход далее. */
    if (largest != idx) {
        void *tmp;
        tmp = ar[idx];
        ar[idx] = ar[largest];
        ar[largest] = tmp;
        heapify(ar, cmp, largest, max);
    }
}
```

Пирамидальная сортировка (2/2)

```
static void buildHeap(void **ar, int (*cmp)(const void *, const void *), int n)
{
    int i;

    for (i = n / 2 - 1; i >= 0; i--)
    {
        heapify(ar, cmp, i, n);
    }
}

void sortPointers(void **ar, int n, int (*cmp)(const void *, const void *))
{
    int i;
    buildHeap(ar, cmp, n);
    for (i = n - 1; i >= 1; i--)
    {
        void *tmp;
        tmp = ar[0];
        ar[0] = ar[i];
        ar[i] = tmp;
        heapify(ar, cmp, 0, i);
    }
}
```

Пирамидальная сортировка

Анализ наихудшего случая

Этап построения:

1. Число сравнений пирамиды глубины $D \leq 2D^2$
2. Глубина пирамиды N элементов $\leq C \log_2 N$

Число узлов на проходе $W(N)$:

$$W(N) = \sum_{i=0}^{D-1} 2(D-i)2^i = \frac{2D(1-2^D)}{1-2} - 2((D-2)2^D + 2) = 2^{D+2} - 2D - 4$$

$$D = \log_2 N$$

$$W(N) = 4N - 2 \log_2 N - 4 = O(N)$$

Этап основного цикла алгоритма:

Выполнение алгоритма при K узлах, по 2 сравнения на узел:

$$W_{loop} = 2 \sum_{k=1}^{N-1} \log_2 K$$

Зная что узлов при глубине $k \sim 2^k$, получаем ($d = \log_2 N$):

$$\begin{aligned} W_{loop} &= 2 \sum_{k=1}^{d-1} k2^k + d(N - 2^d) = 2((d-2)2^d + 2) + d(N - 2^d) = \\ &= 2dN - 2^{d+2} + 4 = 2N \log_2 N - 4N + 4 = O(N \log_2 N) \end{aligned}$$



Быстрая сортировка

01 16 15 09 11 13 14 08 02 10 05 12 06 03 07 04

8 – опорный элемент

Проверка

<8 левая

>8 правая

01 04 15 09 11 13 14 08 02 10 05 12 06 03 07 16

01 04 07 09 11 13 14 08 02 10 05 12 06 03 15 16

01 04 07 03 11 13 14 08 02 10 05 12 06 09 15 16

01 04 07 03 06 13 14 08 02 10 05 12 11 09 15 16

01 04 07 03 06 05 14 08 02 10 13 12 11 09 15 16

01 04 07 03 06 05 02 08 14 10 13 12 11 09 15 16

Наихудший случай $\frac{(N-1)N}{2} = O(N^2)$



Быстрая сортировка

```
void do_qsort(void **ar, int (*cmp)(const void *, const void *), int left, int right)
{
    int pivotIndex;
    if (right <= left)
        return;
    /* Разбиение */
    pivotIndex = selectPivotIndex(ar, left, right);
    pivotIndex = partition(ar, cmp, left, right, pivotIndex);
    if (pivotIndex - 1 - left <= minSize)
        insertion(ar, cmp, left, pivotIndex - 1);
    else
        do_qsort(ar, cmp, left, pivotIndex - 1);
    if (right - pivotIndex - 1 <= minSize)
        insertion(ar, cmp, pivotIndex + 1, right);
    else
        do_qsort(ar, cmp, pivotIndex + 1, right);
}
/** Вызов быстрой сортировки */
void sortPointers(void **vals, int total_elems, int (*cmp)(const void *, const void *))
{
    do_qsort(vals, cmp, 0, total_elems - 1);
}
```

Быстрая сортировка

```
int partition(void **ar, int (*cmp)(const void *, const void *),
             int left, int right, int pivotIndex) {
    int idx,
        store;
    void *pivot = ar[pivotIndex]; /* Перемещаем pivot в конец массива */
    void *tmp = ar[right];
    ar[right] = ar[pivotIndex];
    ar[pivotIndex] = tmp; /* Все значения, не превышающие pivot, перемещаются в начало
    * массива, и pivot вставляется сразу после них. */
    store = left;
    for (idx = left; idx < right; idx++) {
        if (cmp(ar[idx], pivot) <= 0) {
            tmp = ar[idx];
            ar[idx] = ar[store];
            ar[store] = tmp;
            store++;
        }
    }
    tmp = ar[right];
    ar[right] = ar[store];
    ar[store] = tmp;
    return store;
}
```

Быстрая сортировка

Анализ среднего случая

1. Количество сравнений перед разбиением $N - 1$
2. Рекурсивный вызов функций, разбитых на $P - 1$ и $N - P$ элементов

Число узлов на проходе $A(N)$:

$$A(N) = N - 1 + \frac{1}{N} \sum_{i=1}^N (A(i-1) + A(N-i)) = N - 1 + \frac{2}{N} \sum_{i=1}^{N-1} A(i);$$

$$A(1) = A(0) = 0$$

$$N \cdot A(N) = (N-1)N + 2A(N-1) + 2 \sum_{i=1}^{N-2} A(i)$$

$$\begin{aligned} N \cdot A(N) - (N-1) \cdot A(N-1) &= 2A(N-1) + (N-1)N - (N-1)(N-2) = \\ &= 2A(N-1) + 2N - 2 \end{aligned}$$

$$A(N) = \frac{(N+1)A(N-1) + 2N - 2}{N} \approx 1.4(N+1) \log_2 N = O(N \log_2 N)$$



Сортировка слиянием

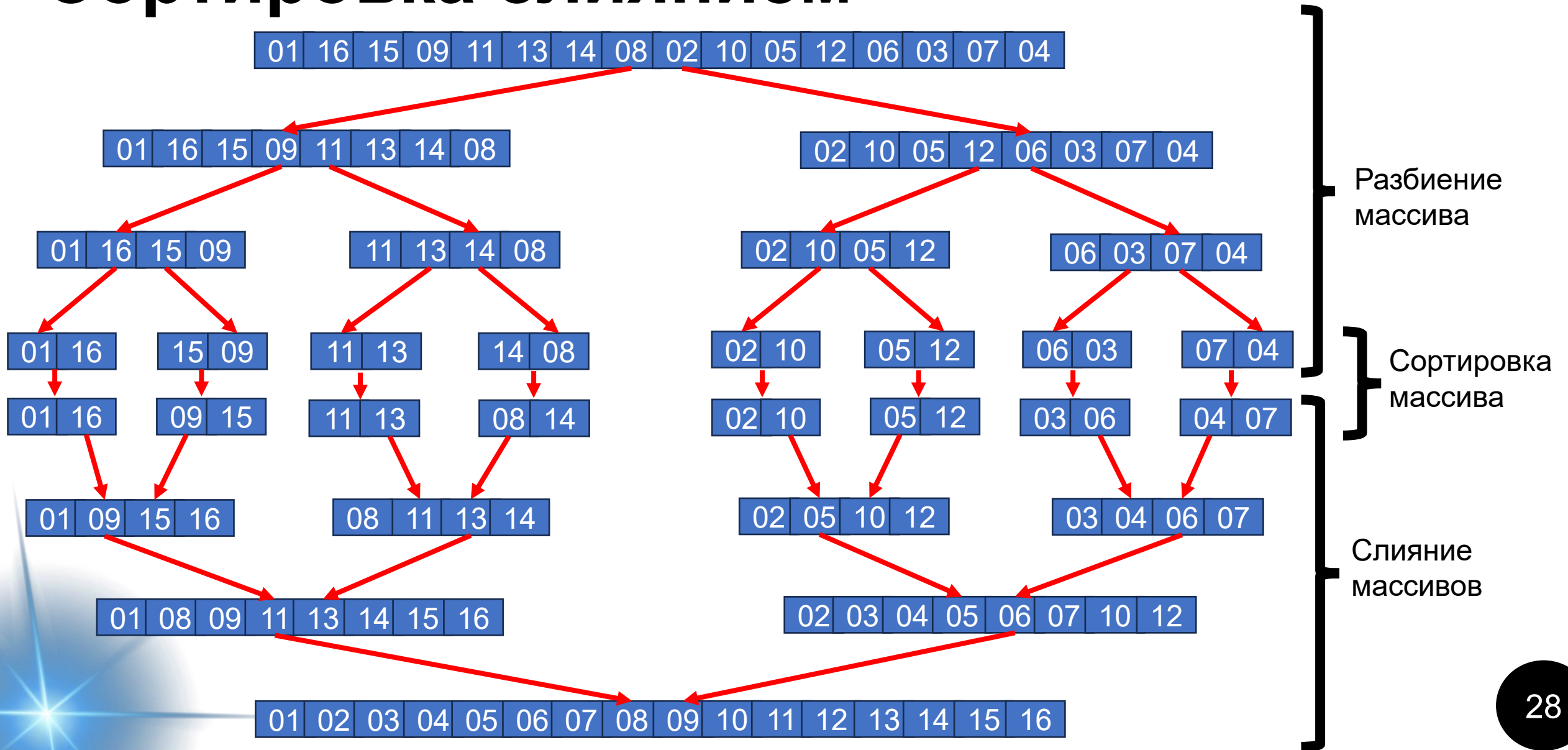
```
void mergesort(void **copy_ar, void **ar, int
(*cmp)(const void *, const void *), int left, int
right)
{
    if (right - left < 2)
        return;
    if (right - left == 2)
    {
        if (copy_ar[left] > copy_ar[left + 1])
        {
            void *temp = copy_ar[left];
            copy_ar[left] = copy_ar[left + 1];
            copy_ar[left + 1] = temp;
        }
        return;
    }
}
```

```
int mid = (right + left) / 2;
mergesort_array(copy_ar, ar, cmp, left, mid);
mergesort_array(copy_ar, ar, cmp, mid, right);
// слияние левой и правой сторон Ar
int i = left, j = mid, idx = left;
while (idx < right)
{
    if (j >= right || (i < mid && ar[i] < ar[j]))
    {
        copy_ar[idx] = ar[i];
        i += 1;
    }
    else
    {
        copy_ar[idx] = ar[j];
        j += 1;
    }
    idx += 1;
}
```

Скорость во всех случаях

$$W(N) = N \log_2 N - N + 1 = O(N \log_2 N).$$

Сортировка слиянием



Как наиболее эффективно
посчитать контрольную сумму
(летящие биты по Wi-Fi)

Самое простое, что приходит

```
uint64_t CountBitsSlow(uint64_t x) { // O(n)
    uint64_t count = 0;
    while (x) {
        count += x & 1u;
        x >>= 1;
    }
    return count;
}
```



Теперь поинтереснее

```
uint64_t CountBitsFast(uint64_t x) { // O(n_1)
    uint64_t count = 0;
    while (x) {
        ++count;
        x &= (x - 1);
    }
    return count;
}
```



Ну как же без треша

```
uint64_t CountBitsFaster(uint64_t x) {  
    x = (x & 0x5555555555555555) + ((x >> 1) & 0x5555555555555555);  
    x = (x & 0x3333333333333333) + ((x >> 2) & 0x3333333333333333);  
    x = (x & 0x0f0f0f0f0f0f0f0f) + ((x >> 4) & 0x0f0f0f0f0f0f0f0f);  
    x = (x & 0x00ff00ff00ff00ff) + ((x >> 8) & 0x00ff00ff00ff00ff);  
    x = (x & 0x0000ffff0000ffff) + ((x >> 16) & 0x0000ffff0000ffff);  
    x = (x & 0x00000000ffffffff) + ((x >> 32) & 0x00000000ffffffff);  
    return x;  
}
```



Можно ли любой сортировкой
сравнениями быстрее $n \log(n)$?

Сортировка подсчетом

Шаг 1 – определяем сколько цифр 1, 2, 3,....

A

2	5	3	0	2	3	0	3
---	---	---	---	---	---	---	---

C

0	1	2	3	4	5
2	0	2	3	0	1

B

--	--	--	--	--	--	--	--

Шаг 2 – определяем сколько раз встречается цифра меньше k

C

0	1	2	3	4	5
2	2	4	7	7	8

Шаг 3 – сортировка

A

2	5	3	0	2	3	0	3
---	---	---	---	---	---	---	---

↑

C

0	1	2	3	4	5
2	2	3	7	7	8

↑

B

0	1	2	3	4	5	6	7
			2				

↑

A

2	5	3	0	2	3	0	3
---	---	---	---	---	---	---	---

↑

C

0	1	2	3	4	5
2	2	3	7	7	7

↑

B

0	1	2	3	4	5	6	7
			2				5

↑

A

2	5	3	0	2	3	0	3
---	---	---	---	---	---	---	---

↑

C

0	1	2	3	4	5
0	2	2	4	7	7

↑

B

0	1	2	3	4	5	6	7
0	0	2	2	3	3	3	5

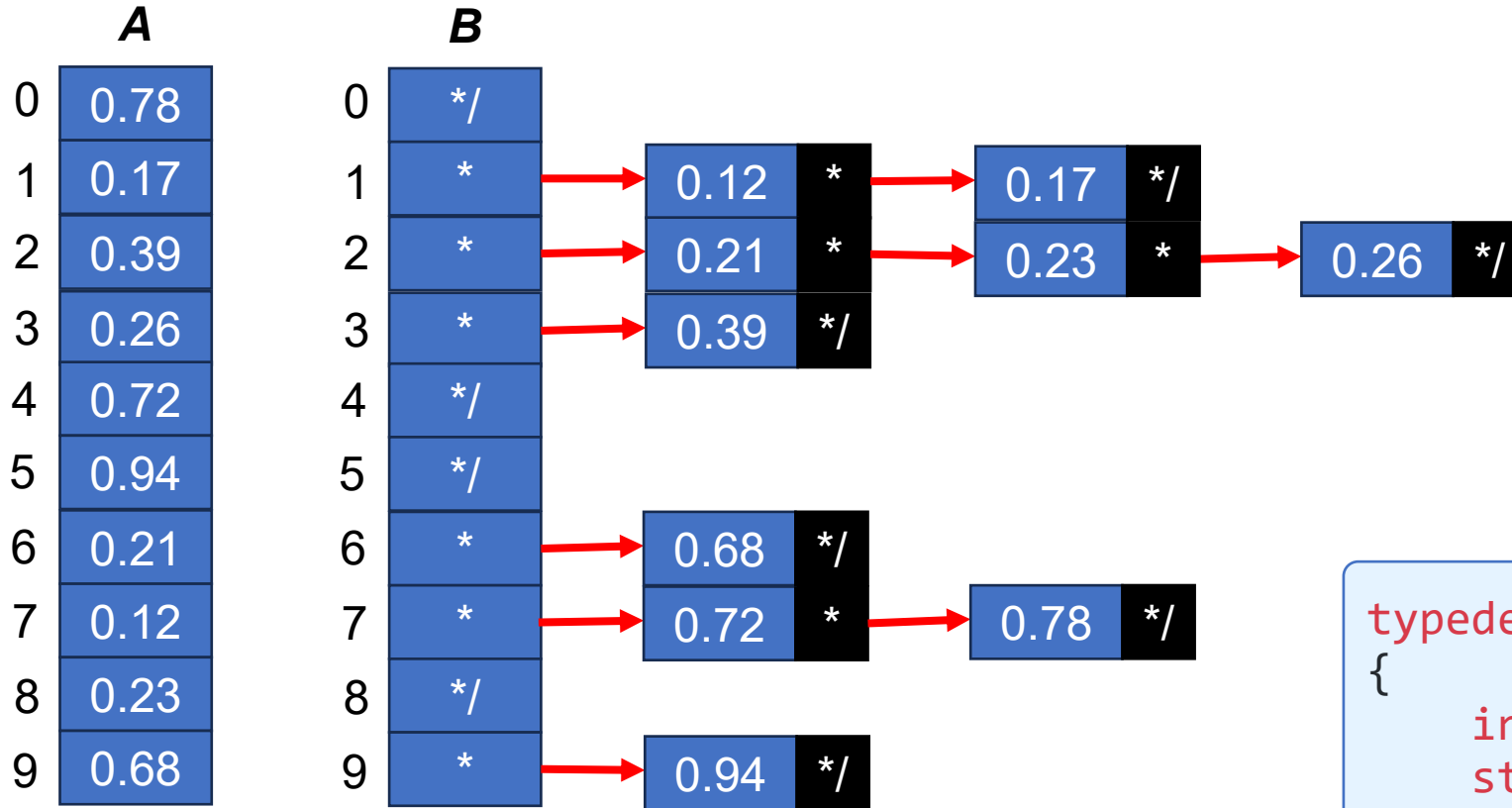
↑

Сортировка подсчетом

```
// Counting-Sort(A, B, k)
void counting_sort(int *A, int *B, int k)
{
    int *C = (int *)calloc(k, sizeof(int)); // Пусть C[0..k] - новый массив, C[i] = 0
    for (size_t j = 0; j < sizeof(A) / sizeof(int); j++)
    {
        C[A[j]] = C[A[j]] + 1; // Сейчас C[i] содержит количество элементов, равных i.
    }
    for (size_t i = 1; i < k; i++)
    {
        C[i] = C[i] + C[i - 1]; // Сейчас C[i] содержит количество элементов, не превышающих i.
    }
    for (size_t j = 0; j < sizeof(A) / sizeof(int); j++)
    {
        B[--C[A[j]]] = A[j];
    }
}
```

Таким образом, общее время составляет $\Theta(k + n)$. На практике обычно сортировка подсчетом применяется, когда $k = O(n)$, и в этом случае общее время работы равно $\Theta(n)$.

Блочная (карманная) сортировка



```
typedef struct node
{
    int data;
    struct node *next;
} list;
list *start = NULL;
list *insert_beg(list *, int);
list **delete_list(list **);
list *sort_list(list *);
```

Блочная (карманная) сортировка

```
// Bucket-Sort(A)
list *merge(list *, int *, int) void bucket_sort(int *A) // A 0 - 1
{
    size_t n = sizeof A / sizeof(int);
    list **B = (list **)malloc(n * sizeof(list *)); // B[0...n - 1] - новый массив
    for (int i = 0; i < n; ++i) // Сделать B[i] пустым списком
        B[i] = NULL;
    for (int i = 0; i < n; ++i) // Вставить A[i] в список B[[nA[i]]]
        insert_beg(roundl(B[n * A[i]]), A[i]);
    for (int i = 0; i < n; ++i)
    { // Отсортировать список B[i]
        sort(B[i]);
    }
    for (int count = 0, i = 0; i < n; ++i)
    {
        merge(B[i], A, &count);
    } // Соединить списки B[0], B[1], ..., B[n - 1] в указанном порядке
}
```


Блочная (карманная) сортировка

Чтобы оценить стоимость этих вызовов, введем случайную величину n_i , обозначающую количество элементов, попавших в карман $B[i]$.

$$T(n) = \theta(n) + \sum_{i=0}^{n-1} O(n_i^2)$$

E – математическое ожидание

$$E(T(N)) = E\left(\theta(n) + \sum_{i=0}^{n-1} O(n_i^2)\right) = \theta(n) + E\left(\sum_{i=0}^{n-1} O(n_i^2)\right) = \theta(n) + \sum_{i=0}^{n-1} O(E(n_i^2))$$

Каждому i карману соответствует одна и та же величина $E(n_i^2)$, поэтому $n_i = \sum_{j=0}^n X_{ij}$

$$\begin{aligned} E(n_i^2) &= E\left(\left(\sum_{j=0}^n X_{ij}\right)^2\right) = E\left(\sum_{j=0}^n \sum_{k=0}^n X_{ij} X_{ik}\right) = \sum_{j=0}^n E(X_{ij}^2) + \sum_{1 \leq j < n} \sum_{\substack{1 \leq k < n \\ k \neq j}} E(X_{ij})E(X_{ik}) = \sum_{j=0}^n \frac{1}{n} + \sum_{1 \leq j < n} \sum_{\substack{1 \leq k < n \\ k \neq j}} \frac{1}{n} \times \frac{1}{n} \\ &= 2 - \frac{1}{n} \end{aligned}$$

$$T(n) = \theta(n) + n * O\left(2 - \frac{1}{n}\right) = \theta(n)$$

Резюме

Сортировка	Время работы	Метод	Область использования
Вставкой	$O(N^2)$	Вставка	Очень малые массивы
Выбором	$O(N^2)$	Двусторонние прохождения, ограничения рассматриваемых пределов	Очень малые массивы
Пузырьковая	$O(N^2)$	Кучи, хранение полных деревьев в массиве	Крупные массивы с неизвестным распределением
Пирамидальная	$O(N \log N)$	Кучи, хранение полных деревьев в массиве	Крупные массивы с неизвестным распределением
Быстрая	$O(N \log N)$ ожидаемое, $O(N^2)$ в худшем случае	«Разделяй и властвуй», перемещение элементов в позицию, рандомизация во избежание худшего случая	Крупные массивы без большого количества дубликатов, параллельная сортировка
Слиянием	$O(N \log N)$	«Разделяй и властвуй», объединение, внешняя сортировка	Крупные массивы с неизвестным распределением, большие объемы данных, параллельная сортировка
Подсчетом	$O(N + M)$	Счет	Крупные массивы с достаточно единообразным распределением значений